

TỐI ƯU HÓA TỰ ĐỘNG MỞ RỘNG ĐA CHIỀU CHO MICROSERVICES TRÊN KUBERNETES SỬ DỤNG HỌC TĂNG CƯỜNG DỰA TRÊN MÔ HÌNH

Lớp: CS2205.CH201

GVHD: PGS. TS. Lê Đình Duy

Trần Bảo Ngọc - 250201019

Tóm tắt

- Lớp: CS2205.CH201
- Link Github của nhóm: [Github](#)
- Link YouTube video: [YouTube](#)



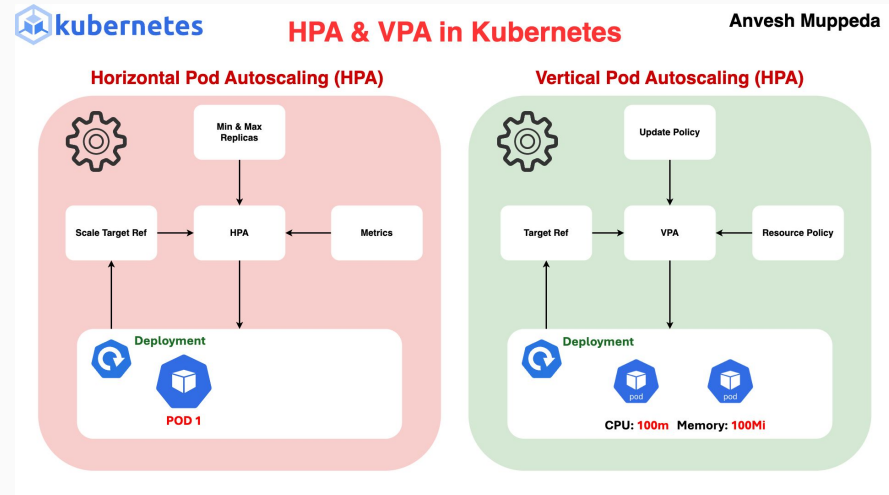
Bối cảnh & Đặt vấn đề

Bối cảnh:

- Kiến trúc Microservices và Kubernetes là tiêu chuẩn hiện nay.
- Thách thức: Cân bằng giữa Hiệu năng (SLA) và Chi phí (Cost).

Vấn đề thực tại:

- Kubernetes HPA/VPA: Phản ứng chậm (Reactive), dựa trên ngưỡng cố định -> "Lag" khi tải tăng đột ngột.
- CoScal: Dùng RL đa chiều rất tốt, NHƯNG là Model-free -> Thời gian training, rủi ro cao khi triển khai thực tế (Cold-start).



Giải pháp & Mục tiêu

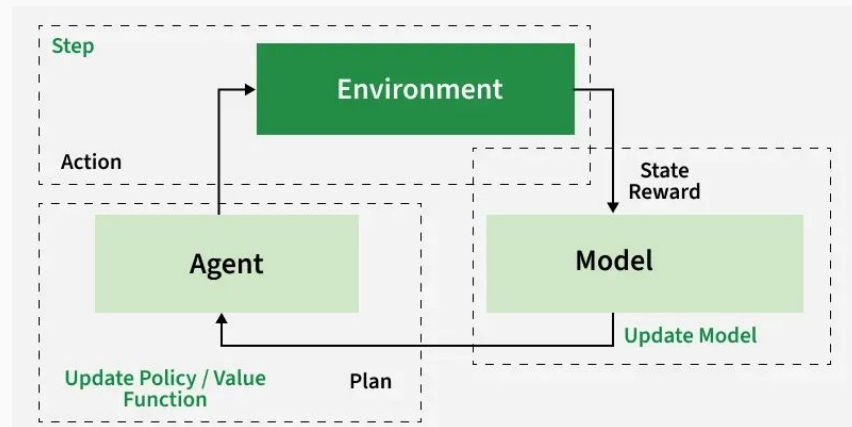
Ý tưởng cốt lõi: Kết hợp sự linh hoạt của CoScal + Tốc độ hội tụ của Model-based RL.

- Tích hợp pha Planning (Quy hoạch): Học trên mô hình giả lập trước khi tác động vào Cluster thật.
- Chuyển từ bị động sang chủ động (Proactive) nhờ dự đoán tải.

Mục tiêu 1: Thiết kế kiến trúc Autoscaler đa chiều trên Kubernetes (Ngang + Dọc + Brownout).

Mục tiêu 2: Xây dựng thuật toán Model-based RL để giảm thời gian hội tụ và giảm rủi ro thử-sai.

Mục tiêu 3: Đánh giá hiệu năng so với HPA mặc định và CoScal gốc.



Kiến trúc hệ thống tổng quan

Nội dung: Giới thiệu mô hình Custom Controller.

Các thành phần chính:

1. Monitor: Thu thập Metrics (Prometheus).
2. Predictor: Dự báo tải (GRU).
3. Agent: Ra quyết định (Model-based RL).
4. Actuator: Thực thi lệnh scale qua K8s API.



Module Dự đoán tải

Công nghệ: Gated Recurrent Unit (GRU).

Tại sao dùng GRU?

- Xử lý tốt dữ liệu chuỗi thời gian (Time-series metrics).
- Nhẹ hơn LSTM nhưng hiệu quả tương đương.

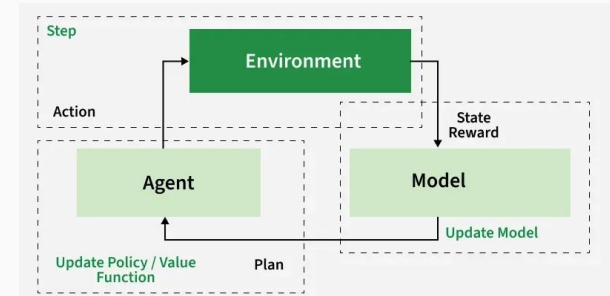
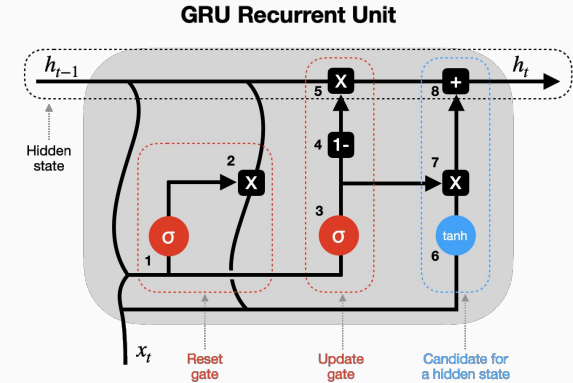
Vai trò: Giúp hệ thống chuẩn bị tài nguyên *trước* khi traffic ập đến.

Thuật toán: Model-based Reinforcement Learning.

$$S \times A \rightarrow S'$$

Quy trình:

- Model Learning: Học động lực học của hệ thống
- Planning: Chạy mô phỏng trên Model ảo để tối ưu chính sách (Policy).
- Execution: Áp dụng hành động tối ưu vào Kubernetes Cluster.

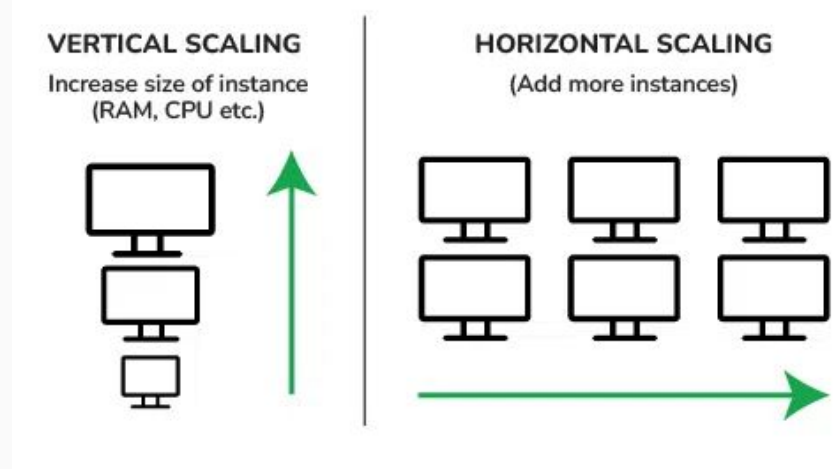


Chiến lược mở rộng đa chiều

Các loại hành động (Action Space):

- Horizontal Scaling: Tăng/giảm số lượng Pod (Bền vững, chậm).
- Vertical Scaling: Tăng/giảm CPU/RAM Limit (Tức thời, giới hạn bởi Node).
- Brownout: Tắt tính năng phụ (Khi quá tải).

Logic: Phối hợp 3 hành động này để đạt hiệu quả cao nhất thay vì chỉ dùng 1 loại như HPA



Kết quả mong đợi

Tốc độ hội tụ: Nhanh hơn 30-40% so với Model-free (CoScal).

SLA: Giảm thiểu vi phạm độ trễ (Response Time) trong các đợt Bursty Traffic.

Tài nguyên: Sử dụng tài nguyên hiệu quả hơn, tránh lãng phí (Over-provisioning).

Kết luận & Tài liệu tham khảo

Kết luận: Đề tài giải quyết bài toán khó về "Cold-start" trong ứng dụng AI vào DevOps.

Hướng phát triển: Mở rộng ra môi trường Multi-cloud hoặc Serverless.

Tài liệu tham khảo:

[1] F. Rossi, M. Nardelli and V. Cardellini, "Horizontal and Vertical Scaling of Container-Based Applications Using Reinforcement Learning," 2019 IEEE 12th International Conference on Cloud Computing (CLOUD), Milan, Italy, 2019, pp. 329-338, doi: 10.1109/CLOUD.2019.00061.

[2] M. Xu et al., "CoScal: Multifaceted Scaling of Microservices With Reinforcement Learning," in IEEE Transactions on Network and Service Management, vol. 19, no. 4, pp. 3995-4009, Dec. 2022, doi: 10.1109/TNSM.2022.3210211.

[3] The Kubernetes Authors. 2024. "Kubernetes Documentation: Horizontal Pod Autoscaling". Available online: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>. Last modified: 2026-01-05, accessed: 2026-01-19.